PYTHON · INFORMATIK

Funktionen ohne Rückgabewert

Programmieren mit Python

Theresa Luternauer & Julia Imhof

HS 2026/27

Kantonsschule Uster

Funktionen ohne Rückgabewert

Grössere Programme werden verständlicher, wenn wir sie in kleine Bausteine aufteilen. Du lernst, eigene Funktionen zu definieren und aufzurufen, Parameter einzusetzen und damit beispielsweise Turtle-Zeichnungen einfacher zu strukturieren.

Worum geht es?

In diesem Kapitel lernst du, längere Programme in kleine, übersichtliche Bausteine zu zerlegen. In Python nennen wir solche Bausteine **Funktionen**.

Du lernst,

- warum Programme in Funktionen unterteilt werden,
- eigene Funktionen zu definieren und aufzurufen,
- Funktionen ohne Parameter zu schreiben,
- Funktionen mit Parametern zu schreiben,
- Turtle-Zeichnungen mithilfe von Funktionen übersichtlich aufzubauen.

Lernziele

Nach diesem Kapitel kannst du …

- erklären, was **modularer Programmentwurf** bedeutet,
- die Vorteile von Funktionen nennen,
- zwischen **Funktionsdefinition** und **Funktionsaufruf** unterscheiden,
- Funktionen ohne Parameter definieren und aufrufen,
- Funktionen mit einem oder mehreren Parametern definieren und aufrufen,
- grössere Turtle-Zeichnungen aus kleineren Funktionen zusammensetzen,
- typische Fehler bei Funktionen erkennen und korrigieren.

---

1. Vom Befehl zur Funktion

Bisher haben wir Programme aus Befehlen aufgebaut, die Python oder die Turtle-Bibliothek bereits zur Verfügung stellen. Zum Beispiel bewegt

```
t.forward(50)
```

die Turtle um 50 Schritte vorwärts.

Pythonkorrekt ist `forward()` eine **Funktion**. Da sie zu unserer Turtle `t` gehört, wird sie genauer als **Methode** bezeichnet. In diesem Kapitel genügt uns zunächst der allgemeinere Begriff **Funktion**.

Wir können auch eigene Funktionen programmieren. Dazu fassen wir mehrere Anweisungen zusammen und geben diesem Programmteil einen passenden Namen.

```
def quadrat():
    for i in range(4):
        t.forward(50)
        t.left(90)
```

Damit haben wir die neue Funktion `quadrat()` definiert. Sie zeichnet ein Quadrat mit der Seitenlänge 50.

!!! note “Vom bisherigen Begriff zum Fachbegriff” Bisher haben wir häufig von **Befehlen** gesprochen. Ab jetzt verwenden wir den Pythonbegriff **Funktion**. Eine Funktion fasst Anweisungen unter einem Namen zusammen und kann später aufgerufen werden.

2. Modularer Programmentwurf

Ein Programm muss manchmal eine schwierige Aufgabe lösen. Statt sofort das ganze Programm zu schreiben, zerlegen wir die Aufgabe in kleinere Teilaufgaben.

Für jede Teilaufgabe schreiben wir eine eigene Funktion. Diese Funktionen sind die **Bausteine** oder **Module** unseres Programms. Anschliessend setzen wir die Bausteine zu einem grösseren Programm zusammen.

Diese Vorgehensweise heisst **modularer Programmentwurf**.

Beispiel: ein Haus aus Bausteinen

Eine Turtle soll ein Haus zeichnen. Das Haus besteht aus einem Quadrat und einem Dach. Wir können deshalb zuerst zwei kleinere Funktionen schreiben:

```
def quadrat():
    for i in range(4):
        t.forward(80)
        t.left(90)


def dach():
    for i in range(3):
        t.forward(80)
        t.left(120)
```

Danach können wir beide Funktionen in einer weiteren Funktion zusammensetzen:

```
def haus():
    quadrat()
    dach()
```

Die Funktion `haus()` verwendet die beiden kleineren Bausteine `quadrat()` und `dach()`.

Vorteile von Funktionen

Funktionen machen Programme …

- **übersichtlicher**, weil eine grosse Aufgabe in kleinere Teile zerlegt wird,
- **verständlicher**, weil Namen wie `quadrat()` oder `dach()` die Aufgabe beschreiben,
- **kürzer**, weil derselbe Programmcode mehrfach verwendet werden kann,
- **leichter veränderbar**, weil ein Baustein nur an einer Stelle angepasst werden muss,
- **weniger fehleranfällig**, weil wiederverwendeter Code nicht immer neu geschrieben wird.

!!! tip “Merke” Eine Funktion sollte eine klar abgegrenzte Aufgabe erfüllen und einen passenden Namen besitzen.

---

3. Funktionen ohne Parameter

Eine Funktion definieren

Eine einfache Funktion wird mit dem Schlüsselwort `def` definiert:

```
def funktionsname():
    anweisung_1
    anweisung_2
```

Eine Funktionsdefinition besteht aus:

1. dem Schlüsselwort `def`,
2. einem selbst gewählten Funktionsnamen,
3. runden Klammern `()`,
4. einem Doppelpunkt `:`,
5. einem eingerückten Funktionskörper.

Der **Funktionskörper** enthält die Anweisungen, die beim Aufruf ausgeführt werden.

```
def linie():
    t.forward(50)
```

!!! warning “Einrückung beachten” Alle Anweisungen, die zur Funktion gehören, müssen eingerückt sein. Die Einrückung zeigt Python, wo der Funktionskörper beginnt und endet.

Eine Funktion aufrufen

Durch die Definition wird die Funktion noch nicht ausgeführt. Sie wird erst ausgeführt, wenn sie aufgerufen wird:

```
linie()
```

Bei einem Funktionsaufruf wird der Funktionsname zusammen mit den runden Klammern geschrieben.

```
import turtle

t = turtle.Turtle()


def quadrat():
    for i in range(4):
        t.forward(50)
        t.left(90)


quadrat()
```

Der Interpreter arbeitet dieses Programm in folgender Reihenfolge ab:

1. Die Turtle-Bibliothek wird importiert.
2. Die Turtle `t` wird erstellt.
3. Python liest die Definition von `quadrat()` und merkt sich die Funktion.
4. Beim Aufruf `quadrat()` springt Python in den Funktionskörper.
5. Die eingerückten Anweisungen werden ausgeführt.
6. Danach wird das Programm unterhalb des Funktionsaufrufs fortgesetzt.

!!! note “Reihenfolge im Programm” Eine Funktion muss definiert sein, bevor sie ausgeführt werden kann. Deshalb steht die Funktionsdefinition normalerweise oberhalb des ersten Funktionsaufrufs.

Eine Funktion mehrmals verwenden

Eine Funktion kann beliebig oft aufgerufen werden:

```
quadrat()
t.forward(100)
quadrat()
```

Der Code zum Zeichnen des Quadrats muss dabei nur einmal programmiert werden.

Funktionen ohne Rückgabewert

Die Funktionen in diesem Kapitel führen eine Tätigkeit aus: Sie bewegen zum Beispiel die Turtle oder zeichnen eine Figur. Sie berechnen keinen Wert, den wir später weiterverwenden möchten. Deshalb bezeichnen wir sie als **Funktionen ohne Rückgabewert**.

Solche Funktionen enthalten keine `return`-Anweisung mit einem Ergebnis.

!!! info “Genauer betrachtet” Python gibt bei einer Funktion ohne ausdrückliches `return` automatisch den speziellen Wert `None` zurück. Für unsere Turtle-Programme ist dieser Wert nicht wichtig. Wir sagen deshalb vereinfachend: Die Funktion hat keinen Rückgabewert.

---

4. Übungen: Funktionen ohne Parameter

Aufgabe 1: Funktionen erkennen

Gegeben ist folgendes Programm:

```
def linie():
    t.forward(50)


def ecke():
    t.left(90)


linie()
ecke()
linie()
```

a) Welche Funktionen stammen aus der Turtle-Bibliothek?
b) Welche Funktionen wurden selbst definiert?
c) Welche Funktionsaufrufe kommen im Hauptprogramm vor?
d) Beschreibe die Bewegung der Turtle in Worten.

Aufgabe 2: Funktionsdefinition und Funktionsaufruf

Gegeben ist:

```
def quadrat():
    for i in range(4):
        t.forward(50)
        t.left(90)


quadrat()
```

a) Markiere die Funktionsdefinition.
b) Markiere den Funktionsaufruf.
c) Wie oft wird `t.forward(50)` ausgeführt?
d) Weshalb wird das Quadrat erst beim letzten Ausdruck gezeichnet?

Aufgabe 3: Programmablauf nachvollziehen

```
def schritt():
    t.forward(30)


def ecke():
    t.left(90)


schritt()
ecke()
schritt()
ecke()
```

a) Notiere die Reihenfolge aller ausgeführten Turtle-Funktionen.

b) Wie oft wird `t.left(90)` ausgeführt?

c) Skizziere den Weg der Turtle.

Aufgabe 4: Eine Funktion ergänzen

Die folgende Funktion soll ein Quadrat zeichnen:

```
def quadrat():
    for i in range(4):
        t.forward(50)
```

a) Welche Anweisung fehlt?

b) Ergänze die Funktion.

c) Rufe die korrigierte Funktion zweimal auf.

Aufgabe 5: Funktionen zusammensetzen

Gegeben sind:

```
def linie():
    t.forward(50)


def ecke():
    t.left(90)
```

Definiere eine neue Funktion `quadrat()`, die nur `linie()` und `ecke()` verwendet. Rufe danach `quadrat()` auf.

Aufgabe 6: Eine Treppe modular zeichnen

Ein Treppenschritt besteht aus einer waagrechten und einer senkrechten Linie:

```
def treppenstufe():
    t.forward(30)
    t.left(90)
    t.forward(30)
    t.right(90)
```

a) Rufe `treppenstufe()` viermal auf.

b) Vereinfache dein Hauptprogramm mit einer Schleife.

c) Definiere zusätzlich eine Funktion `treppe()`, welche die ganze Treppe zeichnet.

d) Erkläre, welche Funktion in diesem Programm als kleinerer Baustein verwendet wird.

Aufgabe 7: Eigene Turtle-Funktion

Definiere eine Funktion für eine der folgenden Figuren:

- ein gleichseitiges Dreieck,
- einen fünfzackigen Stern,
- ein Kreuz,
- einen Pfeil.

Die Funktion soll keine Parameter besitzen. Gib ihr einen Namen, der ihre Aufgabe verständlich beschreibt.

---

5. Funktionen mit Parametern

Die bisherige Funktion `quadrat()` zeichnet immer ein Quadrat mit der Seitenlänge 50:

```
def quadrat():
    for i in range(4):
        t.forward(50)
        t.left(90)
```

Möchten wir verschieden grosse Quadrate zeichnen, könnten wir mehrere fast gleiche Funktionen schreiben. Das wäre jedoch unnötig:

```
def kleines_quadrat():
    # ...


def grosses_quadrat():
    # ...
```

Besser ist eine einzige Funktion, der wir die gewünschte Seitenlänge beim Aufruf mitteilen.

```
def quadrat(seitenlaenge):
    for i in range(4):
        t.forward(seitenlaenge)
        t.left(90)
```

Die Variable `seitenlaenge` in der Funktionsdefinition heisst **Parameter**.

Parameter und Argument

```
def quadrat(seitenlaenge):
    #           ^ Parameter
    for i in range(4):
        t.forward(seitenlaenge)
        t.left(90)


quadrat(60)
#       ^ Argument
```

- Ein **Parameter** ist ein Platzhalter in der Funktionsdefinition.
- Ein **Argument** ist der konkrete Wert, der beim Funktionsaufruf übergeben wird.

Beim Aufruf `quadrat(60)` erhält der Parameter `seitenlaenge` den Wert `60`.

```
quadrat(60)    # zeichnet ein Quadrat mit Seitenlänge 60
quadrat(100)   # zeichnet ein Quadrat mit Seitenlänge 100
```

!!! tip “Merke” Parameter machen eine Funktion flexibel. Derselbe Programmcode kann mit verschiedenen Werten verwendet werden.

Was geschieht beim Aufruf?

Beim Aufruf

```
quadrat(60)
```

geschieht Folgendes:

1. Der Wert `60` wird an die Funktion übergeben.
2. Der Parameter `seitenlaenge` erhält für diesen Aufruf den Wert `60`.
3. Die Funktion verwendet den Wert im Funktionskörper.
4. Nach dem Funktionsaufruf ist der lokale Parameter `seitenlaenge` ausserhalb der Funktion nicht verfügbar.

Der nächste Aufruf kann einen anderen Wert verwenden:

```
quadrat(100)
```

Nun hat `seitenlaenge` während dieses Aufrufs den Wert `100`.

Mehrere Parameter

Eine Funktion kann mehrere Parameter besitzen. Sie werden in den runden Klammern durch Kommas getrennt:

```
def rechteck(breite, hoehe):
    for i in range(2):
        t.forward(breite)
        t.left(90)
        t.forward(hoehe)
        t.left(90)
```

Beim Aufruf muss für jeden Parameter ein Argument angegeben werden:

```
rechteck(100, 50)
```

Die Zuordnung erfolgt von links nach rechts:

Parameter	Argument
`breite`	`100`
`hoehe`	`50`

Die Reihenfolge ist deshalb wichtig:

```
rechteck(100, 50)
rechteck(50, 100)
```

Diese beiden Aufrufe verwenden vertauschte Werte für Breite und Höhe.

Parameter für Farbe und Stiftdicke

Parameter können nicht nur Zahlen, sondern auch andere Werte erhalten:

```
def quadrat(seite, farbe, dicke):
    t.pencolor(farbe)
    t.width(dicke)

    for i in range(4):
        t.forward(seite)
        t.left(90)
```

Aufruf:

```
quadrat(50, "red", 4)
```

Die Argumente werden von links nach rechts zugeordnet:

Parameter	Argument
`seite`	`50`
`farbe`	`"red"`
`dicke`	`4`

!!! warning “Anzahl und Reihenfolge” Beim Aufruf müssen Anzahl und Reihenfolge der Argumente zur Funktionsdefinition passen.

---

6. Übungen: Funktionen mit Parametern

Aufgabe 8: Parameter und Argument erkennen

```
def quadrat(seite):
    for i in range(4):
        t.forward(seite)
        t.left(90)


quadrat(50)
```

a) Wie heisst der Parameter?
b) Welches Argument wird beim Aufruf übergeben?
c) Wo wird der Wert innerhalb der Funktion verwendet?
d) Was müsste geändert werden, damit ein Quadrat mit Seitenlänge 80 entsteht?

Aufgabe 9: Verschiedene Aufrufe

Gegeben ist:

```
def linie(laenge):
    t.forward(laenge)
```

Schreibe drei Funktionsaufrufe, die eine kurze, eine mittlere und eine lange Linie zeichnen.

Aufgabe 10: Zuordnung verstehen

```
def bewegung(laenge, winkel):
    t.forward(laenge)
    t.left(winkel)
```

```
bewegung(100, 90)
```

a) Welchen Wert erhält `laenge`?
b) Welchen Wert erhält `winkel`?
c) Was macht die Turtle zuerst?
d) Was geschieht bei `bewegung(90, 100)`?

Aufgabe 11: Fehler finden

```
def quadrat(seite, farbe):
    t.pencolor(farbe)
    for i in range(4):
        t.forward(seite)
        t.left(90)


quadrat(50)
```

a) Weshalb führt der Aufruf zu einem Fehler?
b) Korrigiere den Aufruf.
c) Formuliere die Fehlermeldung in eigenen Worten.

Aufgabe 12: Argumente vertauscht

```
def linie(laenge, farbe, dicke):
    t.pencolor(farbe)
    t.width(dicke)
    t.forward(laenge)


linie(80, 5, "blue")
```

a) Welche Argumente wurden vertauscht?
b) Schreibe den korrekten Aufruf.
c) Weshalb kann Python nicht selbst erkennen, welche Reihenfolge gemeint war?

Aufgabe 13: Eine Funktion erweitern

Gegeben ist:

```
def dreieck(seite):
    for i in range(3):
        t.forward(seite)
        t.left(120)
```

Erweitere die Funktion um die Parameter `farbe` und `dicke`. Teste sie mit mindestens zwei unterschiedlichen Aufrufen.

Aufgabe 14: Rechteck programmieren

Definiere eine Funktion

```
rechteck(breite, hoehe)
```

die ein vollständiges Rechteck zeichnet. Teste die Funktion mit folgenden Aufrufen:

```
rechteck(100, 50)
rechteck(40, 90)
```

Aufgabe 15: Quadratreihe

Definiere eine Funktion `quadrat(seite)`. Zeichne danach mithilfe mehrerer Funktionsaufrufe Quadrate mit den Seitenlängen 20, 40, 60 und 80.

Überlege, wie die Turtle zwischen den Quadraten verschoben werden muss, damit sie sich nicht überdecken.

Aufgabe 16: Muster aus Funktionen

Definiere zuerst:

```
def quadrat(seite):
    # dein Code
```

Definiere danach eine zweite Funktion:

```
def quadratmuster(seite, anzahl):
    # dein Code
```

Die Funktion `quadratmuster()` soll mehrere Quadrate zeichnen und die Turtle nach jedem Quadrat etwas drehen.

Teste zum Beispiel:

```
quadratmuster(60, 12)
```

Aufgabe 17: Transfer – Haus mit Parametern

Entwickle eine Funktion

```
def haus(groesse):
```

die ein Haus zeichnet. Das Haus soll mindestens aus einem Quadrat und einem dreieckigen Dach bestehen. Verwende innerhalb von `haus()` kleinere Funktionen mit Parametern.

Mögliche Struktur:

```
def quadrat(seite):
    # ...


def dreieck(seite):
    # ...


def haus(groesse):
    # quadrat(...) und dreieck(...) aufrufen
```

!!! question “Denkfrage” Warum ist es für diese Aufgabe besser, `quadrat(seite)` und `dreieck(seite)` zu programmieren, statt die Seitenlängen direkt als feste Zahlen in die Funktionen zu schreiben?

---

7. Typische Fehler

Fehler 1: Doppelpunkt vergessen

```
def quadrat()
    # Doppelpunkt fehlt
```

Korrekt:

```
def quadrat():
    # ...
```

Fehler 2: Funktionskörper nicht eingerückt

```
def quadrat():
t.forward(50)
```

Korrekt:

```
def quadrat():
    t.forward(50)
```

Fehler 3: Klammern beim Aufruf vergessen

```
quadrat
```

Korrekt:

```
quadrat()
```

Fehler 4: Falsche Anzahl Argumente

```
def rechteck(breite, hoehe):
    # ...


rechteck(80)
```

Die Funktion erwartet zwei Argumente, beim Aufruf wird aber nur eines angegeben.

Fehler 5: Funktion zu spät definiert

```
quadrat()


def quadrat():
    # ...
```

Wenn Python den Aufruf erreicht, kennt es die Funktion noch nicht. Die Definition muss vor dem Aufruf ausgeführt werden.

Fehler 6: Funktion nur definiert, aber nicht aufgerufen

```
def quadrat():
    for i in range(4):
```

```
        t.forward(50)
        t.left(90)
```

Dieses Programm merkt sich die Funktion, zeichnet aber noch nichts. Es fehlt der Aufruf:

```
quadrat()
```

---

8. Zusammenfassung

!!! summary “Das Wichtigste in Kürze” - Eine **Funktion** ist ein benannter Programmbaustein. - Mit `def` wird eine Funktion **definiert**. - Mit `funktionsname()` wird eine Funktion **aufgerufen**. - Die Anweisungen im Funktionskörper müssen eingerückt sein. - Funktionen ohne Parameter führen bei jedem Aufruf dieselben Anweisungen aus. - **Parameter** sind Platzhalter in der Funktionsdefinition. - **Argumente** sind konkrete Werte im Funktionsaufruf. - Parameter machen Funktionen flexibel und wiederverwendbar. - Beim modularen Entwurf wird eine grosse Aufgabe in kleinere Funktionen zerlegt. - Die hier verwendeten Funktionen führen Tätigkeiten aus und besitzen keinen für uns wichtigen Rückgabewert.

9. Begriffe

Begriff	Bedeutung
Funktion	Benannter Programmbaustein, der eine bestimmte Aufgabe ausführt
Funktionsdefinition	Festlegung des Namens, der Parameter und des Funktionskörpers mit `def`
Funktionsaufruf	Ausführung einer Funktion durch ihren Namen und runde Klammern
Funktionskörper	Eingerückte Anweisungen innerhalb einer Funktion
Parameter	Platzhalter in der Funktionsdefinition
Argument	Konkreter Wert, der beim Aufruf übergeben wird
Rückgabewert	Ergebnis, das eine Funktion an die aufrufende Stelle zurückgibt
Modularer Entwurf	Zerlegung eines Programms in kleinere, verständliche Funktionen